

Printing the Longest Common Subsequence

DYNAMIC PROGRAMMING TABLE BACKTRACKING GUIDE

The beautiful part about printing the LCS is that after filling the DP table, the table itself contains the decisions made during the computation. We are essentially walking backward through those decisions to reconstruct one optimal subsequence.

Intuition

When computing the table, each cell $dp[i][j]$ stores the length of the LCS between $text1[0 \dots i-1]$ and $text2[0 \dots j-1]$. After reaching $dp[n][m]$, we know the length of the answer, but not the characters.

To recover the characters, we ask: **"How was this value obtained?"**

Three Possibilities

Suppose we are standing at cell (i, j) .

Case 1: Characters match

$text1[i-1] == text2[j-1]$

Then while building the table we used:

$$dp[i][j] = 1 + dp[i-1][j-1]$$

Therefore the current character must belong to the LCS. So:

- Add this character to the answer.
- Move diagonally.

		j-1	j
		+-----+	+-----+
i-1			
		+-----+	+-----+
i			X
		+-----+	+-----+

Move ↖

Example

$text1 = "abcde"$

$text2 = "ace"$

Current characters:

`text1[4] = e`

`text2[2] = e`

Since they match:

`LCS(...e,...e)`

↑

include 'e'

and move to: `(i-1, j-1)`

Case 2: Characters do not match

Suppose `text1[i-1] != text2[j-1]`

Then during table construction we did:

`dp[i][j] = max(dp[i-1][j], dp[i][j-1]);`

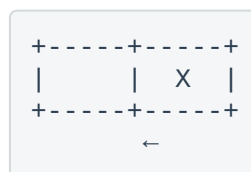
which means one of these two neighbors contributed the answer.

If upper cell is larger (`dp[i-1][j] > dp[i][j-1]`): then the optimal answer came from `(i-1, j)`. Move upward.



because character `text1[i-1]` wasn't used.

If left cell is larger (`dp[i][j-1] > dp[i-1][j]`): then move left.



because character `text2[j-1]` wasn't used.

Why start from the last cell?

`dp[n][m]` represents the answer for the entire strings: `text1[0...n-1]` and `text2[0...m-1]`. Since the answer was built from smaller subproblems, tracing backwards naturally reveals which characters participated. Think of it like following breadcrumbs back to the start.

Visualization

Take:

```
text1 = abcde
```

```
text2 = ace
```

	""	a	c	e
""	0	0	0	0
a	0	1	1	1
b	0	1	1	1
c	0	1	2	2
d	0	1	2	2
e	0	1	2	3

Start at: `dp[5][3]=3`

Current characters: `e == e`

Include: `answer = "e"`

Move diagonally: `dp[4][2]`

At: `d != c`

Compare: `dp[3][2]=2` and `dp[4][1]=1`. Upper is larger. Move up.

At: `c == c`

Include: `answer = "ec"`

Move diagonally.

At: `b != a`

Compare neighbors: `dp[1][1]=1` and `dp[2][0]=0`. Move up.

At: `a == a`

Include: `answer = "eca"`

Move diagonally to (0,0).

Why is the answer reversed?

Notice the order of collection: `e` → `ec` → `eca`, because we are traveling from the end of the strings toward the beginning. Therefore:

```
reverse(answer.begin(), answer.end());
```

gives: `ace`

The fundamental intuition

While filling the table, every cell answers: *"What is the length of the best LCS up to here?"*

During backtracking, we ask: *"Which previous state produced this answer?"*

- **Match** → diagonal ↖ and take the character.
- **No match** → follow the neighbor having the larger value.
- Continue until reaching row 0 or column 0.
- Reverse the collected characters.

Documentation comment

```
/*  
Backtracking through the DP table reconstructs one Longest Common  
Subsequence.  
  
Starting from dp[n][m], each cell represents the answer for prefixes  
text1[0...i-1] and text2[0...j-1].  
  
1. If text1[i-1] == text2[j-1], this character contributed to the LCS.  
   Add it to the answer and move diagonally to (i-1,j-1).  
  
2. Otherwise, the current value came from either the upper cell  
   dp[i-1][j] or the left cell dp[i][j-1]. Move toward the cell with  
   the larger value, since that path produced the optimal subsequence.  
  
Since traversal proceeds from the end of the strings toward the  
beginning, characters are collected in reverse order. Reverse the  
result at the end to obtain the correct LCS.  
*/
```

This "follow the parent state" idea is the same reconstruction technique used in Longest Palindromic Subsequence, Shortest Common Supersequence, Edit Distance operations, LIS reconstruction, Knapsack item selection, and many other DP problems.